

Introduzione al linguaggio c++

Edoardo Lo Iacono

Indice

1	Perché studiamo c++?	3
2	Installazione su visual studio code	3
3	Sintassi di base	3
3.1	std::cout	4
3.2	reutrn 0	4
4	La libreria iostream	4
4.1	Concetto generale di libreria	4
4.2	La libreria	4
4.3	Gli oggetti	4
4.3.1	Chiedere un valore all'utente e stamparlo	5
4.4	Gli operatori di flusso	5
5	Il blocco condizionale	5
5.1	Lo switch	6
6	I cicli	7
6.1	Il ciclo for	7
6.2	Il ciclo while	7
6.3	Il ciclo do-while	7

1 Perché studiamo c++?

C++ è un linguaggio di programmazione creato da *Bjarne Stroustrup*, entrato in vigore nel 1985. Nasce come estensione del linguaggio c e con il tempo si espande notevolmente rispetto ad esso. Nonostante sia un po' datato è uno dei linguaggi di programmazione più completi ed utilizzati ancora al giorno d'oggi, senza saperlo lo troviamo nella maggior parte delle applicazioni che usiamo quotidianamente; sono infatti sviluppati (completamente o in parte) in c++ applicazioni come:

- I principali web browser (Chrome, Firefox, Edge)
- I programmi del pacchetto office
- Photoshop
- Diversi videogiochi con il motore grafico unreal engine

2 Installazione su visual studio code

Per lavorare con c++ possiamo usare diversi IDE, alcuni sono dedicati ai linguaggi c come per esempio [codeblocks](#), nulla vieta di usare questo o altri ambienti di sviluppo. Vediamo però di seguito come fare programmi in c++ tramite visual studio code: IDE molto versatile e professionale grazie alla vastità di estensioni disponibili.

1. Installare visual studio code al link: [vscode](#)
2. Scaricare il compilatore seguendo le istruzioni al seguente link: [link](#)
3. Una volta installato aprire a sinistra la barra delle estensioni
4. Cercare nella barra di navigazione "c++" ed installare le estensioni "c++" e "c++ Extension Pack" le quali ci permetteranno non solo di eseguire il nostro codice ma anche di vedere a schermo eventuali errori e, come negli altri linguaggi un color coding che ci permetta di capire l'entità di tutti i nostri elementi.
5. Cercare nella barra di navigazione "code runner" e scaricarlo

3 Sintassi di base

Prendiamo in esame il più basilare dei codici in c++ per iniziare a parlare della sintassi di base.

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello World!";
5     return 0;
6 }
```

Lasciamo per un momento da parte la riga `'include <iostream>` che approfondiremo meglio in seguito parlando di librerie.

La funzione `main()` viene comunemente chiamata **entry point**, e racchiude le prime istruzioni che verranno eseguite quando viene fatto partire il programma, è importante specificare il fatto che sia

la prima in quanto potremmo avere (come vedremo meglio in seguito) diverse funzioni nel nostro programma, il main viene sempre eseguito prima di ogni altra funzione!

3.1 `std::cout`

Iniziamo a parlare del codice vero e proprio, il primo comando che vediamo è **cout**. `cout` è un oggetto che fa parte della libreria standard (`std`), per questo andiamo a indicarla, ci permette di **mandare a schermo un testo**. Lo "schermo" su cui viene stampato il nostro testo prende il nome di *console*, infatti `cout` non è altro che l'acronimo per Console Out.

Nulla di trascendentale come implicazione logica ma si noti bene una cosa che tornerà nella maggior parte dei comandi che vedremo, quando il codice *esegue* in maniera pragmatica qualcosa (manda a schermo, salva in memoria) è necessario terminare il comando con `;`, questa è una regola sempreverde nei linguaggi di programmazione basati su `c` ma non in tutti i linguaggi, il `;` serve al compilatore per capire che quell'operazione è finita e passare alla prossima.

3.2 `return 0`

Notiamo a riga 5 questo `return 0`, codice che al momento come logica non ci dice nulla ma che ci permette di aprire un discorso ben più ampio (e maggiormente approfondito nel file "fondamenti di programmazione") per quanto riguarda variabili, tipi e funzioni tipizzate. Prendiamo ora per buono il fatto che questo `return 0` ci stia semplicemente dicendo che il codice è stato eseguito con successo e senza errore. Troviamo dopo il `return` un numero in quanto abbiamo dichiarato la funzione come intero, in questo caso ci aspettiamo che ritorni un valore intero (convenzionalmente 0 quando va tutto bene e -1 quando qualcosa va storto).

4 La libreria `iostream`

4.1 Concetto generale di libreria

Quando in informatica parliamo di **librerie** ci riferiamo a **interi blocchi di codice messi a nostra disposizione per l'uso**, è necessario importare librerie diverse per compiere azioni diverse, approfondiamo di seguito la libreria basilare e fondamentale: `iostream`

4.2 La libreria

La libreria `iostream` è alla base della programmazione in `c++`, il suo nome esteso significa letteralmente **Input/Output Stream** che possiamo tradurre come flusso di input ed output, la sua funzione è proprio quella di far **interagire delle azioni in input con delle risposte in output** mettendo a nostra disposizione dei metodi concreti per far interagire il nostro programma con l'esterno

4.3 Gli oggetti

Quando viene importata questa libreria si ha l'accesso a quattro oggetti fondamentali, andiamo a vederli meglio di seguito:

Oggetto	Funzione	Direzione
std::cout	Permette di stampare dati sulla console	Uscita
std::cin	Legge dati dalla console	Entrata
std::cerr	Manda sulla console un messaggio di errore	Uscita
std::clog	Alternativo a cerr	Uscita

4.3.1 Chiedere un valore all'utente e stamparlo

```

1  #include <iostream>
2
3  int main(){
4      int n;
5      std::cout << "Inserisci un valore: ";
6      std::cin >> n;
7      std::cout << "Hai inserito il valore: " << n;
8  }
```

4.4 Gli operatori di flusso

Notiamo nell'esempio sopra citato la presenza di operatori quali << e >>. Cerchiamo di comprendere meglio in seguito il loro utilizzo per poter rendere questo codice più logico e comprensibile:

- << è un operatore di inserimento: Quello che viene inserito dopo l'operatore viene messo all'interno di quello che lo precede
- Con la stessa logica si usa l'operatore >> che consideriamo però come un operatore di estrazione in quanto mettiamo prima quello che viene inserito e dopo il luogo in memoria in cui viene inserito

5 Il blocco condizionale

Il blocco condizionale in c++ rimane invariato rispetto agli altri linguaggi evoluti da c, troviamo infatti una sintassi del seguente tipo:

```

1  if(condizione)
2  {
3      //Codice eseguito se la condizione vera
4  }
5  else if(condizione)
6  {
7      //Codice eseguito se la prima condizione falsa e la seconda vera
8  }
9  else
10 {
11     //Codice eseguito se nessuna delle precedenti vera
12 }
```

Per una panoramica maggiore sul funzionamento dei blocchi condizionali si veda il file *Fondamenti di programmazione* al cap.4.

5.1 Lo switch

Quando ci capita di avere più condizioni annidate l'una dentro l'altra possiamo avvalerci dell'istruzione `switch`, vediamo con un esempio:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int giorno = 3;
6      switch (giorno) {
7          case 1:
8              cout << "Lunedì";
9              break;
10         case 2:
11             cout << "Martedì";
12             break;
13         case 3:
14             cout << "Mercoledì";
15             break;
16         default:
17             cout << "Altro_giorno";
18             break;
19     }
20     return 0;
21 }
```

Prima di vedere un po' meglio le caratteristiche dello `switch` è importante notare come saremmo potuti giungere allo stesso risultato anche grazie ad una sequenza di `if` ed `else if` che sarebbe però risultata molto più confusionaria alla lettura.

Analizzando il codice, possiamo distinguere gli elementi fondamentali che compongono questa struttura:

- **L'espressione:** Tra parentesi tonde dopo lo `switch` inseriamo la variabile da controllare (in questo caso `giorno`).
- **I casi (case):** Rappresentano i possibili valori che la variabile può assumere. Se il valore coincide, viene eseguito il codice corrispondente.
- **L'istruzione break:** È fondamentale perché "interrompe" lo `switch`. Senza di essa, il programma continuerebbe a eseguire tutti i casi successivi fino alla fine della struttura.
- **Il caso default:** Funziona come l'ultimo `else` di una catena di `if`; viene eseguito se nessuno dei casi precedenti è stato soddisfatto.

In sintesi, lo `switch` rende il codice più pulito e facile da mantenere, specialmente quando dobbiamo gestire un numero elevato di alternative basate su un'unica variabile.

6 I cicli

6.1 Il ciclo for

Anche per quanto riguarda i cicli, la sintassi del c++ rimane invariata rispetto ai suoi linguaggi di programmazione predecessori, vediamo di seguito la sintassi del ciclo for:

```
1  for(int i = 0; i < n; i++)
2  {
3      operazioni
4  }
```

ricordiamo come il ciclo for ci permetta di eseguire, per un numero noto di volte una stessa porzione di codice, in particolare il for è costituito da 3 parti: la **variabile di ciclo** (in questo caso `int i = 0`), la **condizione di uscita** che ci permette di comprendere se rimanere dentro il ciclo oppure uscire e continuare con il codice quando risulta falsa (qui `i < n`) e l'**operazione sulla variabile** che in questo caso ci permette semplicemente di incrementare `i` ad ogni giro. I valori presentati qui sono ovviamente del tutto arbitrari!!

6.2 Il ciclo while

A differenza del ciclo for, il ciclo **while** viene utilizzato quando non conosciamo a priori il numero di iterazioni da compiere. Il codice viene eseguito finché una determinata condizione rimane vera:

```
1  while(condizione)
2  {
3      // operazioni
4  }
```

In questa struttura, la condizione viene controllata **prima** di eseguire le operazioni. Se la condizione è falsa già all'inizio, il corpo del ciclo non verrà eseguito nemmeno una volta. È fondamentale che all'interno del ciclo avvenga qualcosa che modifichi la condizione, per evitare di finire in un loop infinito.

6.3 Il ciclo do-while

Il ciclo **do while** è una variante del precedente, con una differenza piccola ma sostanziale nella logica di esecuzione:

```
1  do
2  {
3      // operazioni
4  } while(condizione);
```

In questo caso, il controllo della condizione avviene **dopo** l'esecuzione del blocco di codice. Questo garantisce che le operazioni vengano eseguite almeno una volta, indipendentemente dal fatto che la condizione sia vera o falsa.